

Chapter 7: Credentials in Object Properties

1 The Concept: Storing Secrets in Object Metadata

Technical Concept: Unintended Read Access to User Properties

Active Directory objects, such as users, contain numerous properties including account status, office location, and expiration dates.

- **The Misconception:** Administrators often store cleartext passwords in properties like **Description** or **Info**, mistakenly believing only other administrators can view these fields.
- **The Reality:** By default, **every authenticated domain user** can read the majority of object properties, including those often abused for credential storage.
- **The Vulnerability:** This exposure allows any compromised account to "scrape" the entire directory for passwords hidden in metadata fields.

The Grand Hotel Analogy: The Staff ID Badges

Imagine every staff member at the hotel wears a **Standard ID Badge** (*User Object*).

- **The Badge Info:** The front of the badge shows their name and department. The back of the badge has a "Notes" section (*Description/Info Property*).
- **The Mistake:** To avoid forgetting it, an **HR Manager** writes a new employee's temporary room-access code (*Password*) on the back of their badge.
- **The False Security:** The HR Manager assumes only supervisors will ever check the back of a badge.
- **The Exposure:** However, anyone in the hotel even a **Dishwasher** (*Bob*) can simply ask to see a colleague's badge and read the code written on the back.

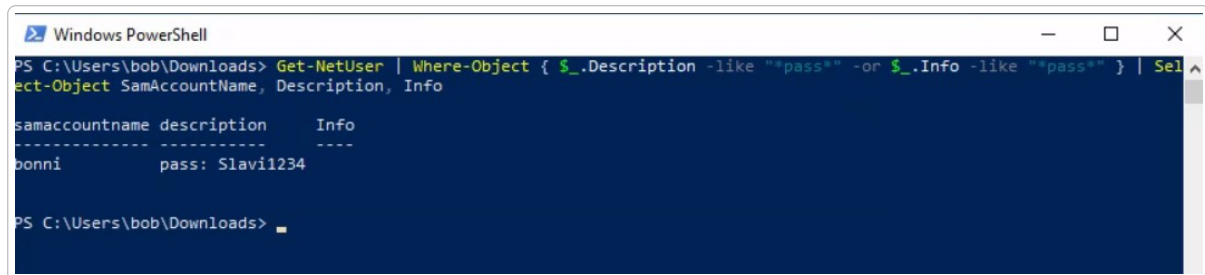
2 Attack Execution: Step-by-Step

2.1 Step 1: Enumerating Object Properties

To identify these exposures, we can use queries like the one below to enumerate the entire domain for specific search terms or strings hidden within the **Description** or **Info** fields.

>_ Terminal: Windows PowerShell (as bob)

```
Get-NetUser | Where-Object { $_.Description -like "*pass*" -or $_.Info -like "*pass*" } | Select-Object SamAccountName, Description, Info
```



The screenshot shows a Windows PowerShell window with the following content:

```
PS C:\Users\bob\Downloads> Get-NetUser | Where-Object { $_.Description -like "*pass*" -or $_.Info -like "*pass*" } | Select-Object SamAccountName, Description, Info
```

samaccountname	description	Info
bonni	pass: Slavi1234	

```
PS C:\Users\bob\Downloads>
```

The results of this query identify the following sensitive information exposed in the domain metadata:

- **Target User:** bonni
- **Discovery Property:** Description
- **Exposed Credential:** Slavi1234

This information can be used immediately to attempt unauthorized authentication against domain resources.

3 The Impact: Discovery vs. Authentication

Identifying a password in a property field is only the first step. The attacker must then verify if the credential is still valid and what level of access it provides.

Technical Concept: Verification and Movement

- **Validation:** Attempting to log in with the discovered password triggers authentication logs (Event IDs 4624/4625).
- **Privilege Check:** If the account (like 'bonni') has elevated rights or access to sensitive servers (like DC1), the breach escalates immediately.

4 Detection & Prevention

Technical Concept: Defense Strategies

- **Prevention (Education):** Train administrators and privileged users to never store credentials in object metadata.
- **Automation:** Use automated scripts for user creation to eliminate the human error of manual field entries.
- **Detection (Baselining):** Monitor for abnormal login behavior from accounts whose credentials may have been exposed. Note: Event ID 4738 (Object Modified) does **not** show which specific property was changed, making it hard to detect the initial storage of the password.
- **Honeypots:** Create a fake service account (e.g., `svc-iis`) with an incorrect password in the Description field. Any failed login attempt (Event ID 4625) with that fake password is a guaranteed indicator of an attacker scraping properties.

Log Analysis Task

The TargetSid for the user 'bonni': _____
(Check Event Viewer logs on DC1 for Event ID 4768/4624 to find this value)